

Optimización de CNN de clasificación con herramientas HPC

Andrés Mardones Domcke

andres.mardones@alumnos.uach.cl

Abstract

Este trabajo compara cinco implementaciones de una red neuronal convolucional (CNN) de clasificación utilizando distintas técnicas de computación de alto rendimiento (HPC): Python base, vectorización con NumPy, paralelización en CPU con Cython/OpenMP, aceleración en GPU mediante JAX y computación distribuida con Ray. Utilizando el dataset CIFAR-10, se evaluaron rigurosamente los tiempos de ejecución de la convolución, el forward pass y el entrenamiento completo, además de medir el consumo energético y los picos de memoria de cada entorno. Los resultados demuestran que NumPy obtuvo la mayor aceleración con operaciones vectorizables (superando las 1000 veces en la convolución aislada), el perfil físico más eficiente en energía y de los que menos memoria utilizan. El estudio confirma el impacto crítico de las estrategias HPC en la reducción de tiempos para CNNs, concluyendo que la optimización algorítmica puede llegar a ser más decisiva que la paralelización masiva por software.

Palabras clave: CNN, HPC, OpenMP, Cython, JAX, CPU, GPU, paralelización, RAY, clasificación de imágenes.

1. Introducción

El reciente crecimiento de la visión por computador ha sido muy relevante en la industria de la informática y la inteligencia artificial. Existen muchas aplicaciones que aportan soluciones en diversas áreas, tales como medicina, automatización industrial, deporte, astronomía, entre otras. Dentro de esta rama existe un apartado dedicado exclusivamente a las redes neuronales, técnica por excelencia en el estado del arte para dar solución a tareas como extracción de características, detección, tracking, segmentación o clasificación.

Sin embargo, el éxito de las redes neuronales convolucionales trae consigo un claro problema: el coste computacional. Y es que muchos aspectos inherentes al funcionamiento de las redes provocan que el procesamiento sea muy grande. Por ejemplo, la arquitectura de la red, es decir, la cantidad de filtros, de capas (profundidad), o el tamaño de las imágenes a procesar. Al aumentar estos parámetros, los

tiempos de ejecución pueden aumentar considerablemente. Además, para obtener buenos resultados, ya sea para aumentar la precisión de la red o para evitar el sobreajuste, se puede requerir de aumentar el tamaño del dataset o de entrenar por más épocas, incrementando el coste computacional.

Es por estas razones que resulta lógico buscar soluciones relacionadas a optimizar las operaciones internas de las redes neuronales convolucionales con tal de atenuar el crecimiento de los tiempos de ejecución a medida que las redes escalan en alguno de sus parámetros. Y es aquí donde cobra importancia la computación de alto rendimiento, un área que ofrece herramientas pensadas para reducir los tiempos de ejecución y utilizar los recursos de manera eficiente en problemas complejos.

El objetivo de este informe es comparar y analizar de forma justa 5 implementaciones de una red neuronal convolucional, con el fin de obtener conclusiones acerca de cuál enfoque puede resultar más adecuado dada la naturaleza de los cálculos que se realizan en un problema como este. Esas implementaciones son:

- Versión con operaciones base de Python
- Versión mejorada utilizando vectorización con NumPy
- Versión paralela en CPU usando Cython
- Versión paralela en GPU usando JAX.
- Versión distribuida usando RAY.

A continuación se procede a explicar la naturaleza del problema, así como la implementación de las versiones y los resultados de las comparaciones entre ellas.

2. Trabajos relacionados

La optimización de Redes Neuronales Convolucionales (CNNs) ha evolucionado desde implementaciones puramente secuenciales hacia el aprovechamiento de herramientas de Computación de Alto Rendimiento (HPC). La literatura divide los esfuerzos de optimización en varias estrategias de abstracción y paralelismo, las cuales se alinean con las variantes evaluadas en este estudio. Chellapilla et al. (2006) [3] demostraron que la reestructuración matemática

de la convolución espacial, transformándola en un producto de matrices a través del operador `im2col` es una estrategia fundamental en HPC. Esta técnica elimina el costoso bucle espacial y maximiza la utilización de la caché de la CPU mediante accesos continuos a memoria, sirviendo de base para los enfoques modernos de vectorización y paralelismo.

Cuando la vectorización de alto nivel es insuficiente debido a las restricciones de los lenguajes interpretados, la literatura apunta hacia la compilación estática y el paralelismo a nivel de hilos (thread-level parallelism). En el ecosistema de Python, Cython [1] ha sido validado como una herramienta eficaz para la computación científica al permitir la traducción de código a C puro con tipado estático y, fundamentalmente, la liberación del Global Interpreter Lock (GIL).

El paradigma moderno de HPC para el entrenamiento de redes neuronales está dominado por el uso de aceleradores de hardware como las Unidades de Procesamiento Gráfico (GPUs). Trabajos recientes como el de Zhao et al. [5] exploran el diseño de frameworks paralelos para maximizar el paralelismo espacial del hardware. Históricamente, este nivel de optimización requería la escritura de kernels nativos complejos (ej. en NVIDIA CUDA). Sin embargo, la aparición de frameworks de nueva generación como JAX [2] ha transformado este flujo.

La mayoría de los estudios HPC en Deep Learning, como el desarrollado por Rahman et al. (2024) [4], miden el tiempo global de entrenamiento integrando herramientas como CUDA u OpenMPI dentro de frameworks robustos y opacos. Este trabajo se separa de dicha línea al plantear un enfoque incremental de "caja blanca" basado en la equivalencia funcional. Manteniendo una arquitectura de red y un dataset idénticos (CIFAR-10), se experimenta el rendimiento computacional de manera pura. La diferencia clave está en el control del algoritmo: desde bucles explícitos en Python y vectorización en NumPy, hasta el control de hilos en CPU con Cython y la optimización JIT en GPU mediante JAX. El objetivo final no es la precisión del modelo, sino obtener el speedup exacto de cada solución.

3. Solución propuesta y metodología

Para comparar los diferentes métodos se eligió una arquitectura de red neuronal convolucional de clasificación de imágenes con las siguientes características:

- 1 capa convolucional con 8 filtros 3x3x3
- Función de activación ReLU
- Capa de reducción MaxPooling 2x2
- 1 capa fully connected con 10 neuronas
- Softmax

3.1. Versión Python base

Esta versión se incluye con el fin de analizar el rendimiento de la implementación más básica, estableciendo el piso sobre el cual mejorar. Para eso se decide utilizar Python puro, lo que implica ejecutar todo de manera secuencial, anidando bucles for para las operaciones incluidas en la red.

Algorithm 1 Convolución en Python puro

```

1: Entrada: Imagen  $X \in \mathbb{R}^{H \times W}$ , filtro  $W \in \mathbb{R}^{K \times K}$ 
2: Salida: Mapa de activación  $Y$ 
3:  $output\_height \leftarrow H - K + 1$ 
4:  $output\_width \leftarrow W - K + 1$ 
5: Inicializar  $Y$  con ceros
6: for  $i = 0$  to  $output\_height - 1$  do
7:   for  $j = 0$  to  $output\_width - 1$  do
8:      $suma \leftarrow 0$ 
9:     for  $m = 0$  to  $K - 1$  do
10:      for  $n = 0$  to  $K - 1$  do
11:         $suma \leftarrow suma + X[i + m][j + n] \cdot$ 
            $W[m][n]$ 
12:      end for
13:    end for
14:     $Y[i][j] \leftarrow suma$ 
15:  end for
16: end for
17: return  $Y$ 

```

En este pseudocódigo de la convolución con Python puro se aprecia el coste computacional del algoritmo como tal, forzando a incluir 4 bucles for anidados, además accediendo constantemente a memoria para calcular las sumas y multiplicaciones. Este patrón se repite constantemente en las demás partes de la red, puesto que tanto ReLU, como MaxPooling, SoftMax y la capa Fully Connected necesitan recorrer arreglos.

3.2. Versión vectorizada con NumPy

La segunda versión implementada corresponde a mejoras con respecto a la vectorización de las operaciones. A diferencia de Python puro, donde gran parte de las operaciones se ejecutan mediante ciclos explícitos interpretados uno a uno, NumPy agrupa múltiples cálculos sobre arreglos completos y los delega a rutinas internas implementadas en lenguaje compilado. Esto disminuye el costo asociado al intérprete de Python y aprovecha mejor la memoria y el procesador. Además, muchas de estas operaciones internas se encuentran optimizadas para trabajar sobre bloques contiguos de datos y utilizar instrucciones vectoriales del hardware.

Un ejemplo de esta mejora viene con la convolución, donde el procesamiento se reduce notablemente dada la

condición vectorial de la operación:

Algorithm 2 Convolución vectorizada con NumPy

```
1: Entrada: Imagen  $X$ , filtros  $W$ 
2: Salida: Mapa de activación  $Y$ 
3:  $windows = sliding\_window\_view(X, (3, 3))$ 
4:  $windows = windows.transpose(0, 1, 3, 4, 2)$ 
5:  $Y = np.tensordot(windows, W)$ 
6: return  $Y$ 
```

En el código de esta versión para la convolución no se ve ningún bucle for, todas las operaciones se producen de forma vectorizada. Comenzando por `sliding_window_view()`, una función que genera todas las ventanas de la imagen para aplicarles convolución. *Transpose*, reorganiza los ejes y *tensordot* es la encargada de multiplicar las ventanas con los filtros de forma simultánea. Todas estas funciones, al pertenecer a la librería de NumPy, cuentan con una optimización elevada, principalmente, debido a que están construidas en C. Cabe mencionar que todas las demás etapas de la red fueron también optimizadas con funciones de NumPy, utilizando operaciones como `np.max()` o `np.sum()`

3.3. Versión paralelizada en CPU con Cython

Si bien existen muchas formas de paralelizar una red neuronal convolucional, se decidió por aplicarla solo en la convolución, sobre la dimensión de filtros, distribuyendo el cálculo de cada filtro entre distintos hilos mediante la instrucción `prange`. De esta manera, cada hilo ejecuta de forma independiente la convolución correspondiente sobre toda la imagen para un subconjunto de filtros, calculando en paralelo sus mapas de activación. Además del paralelismo, se utilizó tipado estático y vistas de memoria en Cython, reduciendo el costo asociado al intérprete de Python y mejorando el acceso a memoria. Debido a que el cálculo de cada filtro es independiente del resto, esta estrategia permite aprovechar eficientemente la arquitectura multicore de CPU y reducir el tiempo total de ejecución de la capa convolucional.

Algorithm 3 Convolución paralela en CPU con Cython

```
1: Entrada: Imagen  $X$ , filtros  $W$ 
2: Salida: Mapa de activación  $Y$ 
3: Inicializar  $Y$ 
4: for en paralelo cada filtro  $f$  do
5:   for cada posición  $(i, j)$  de la imagen do
6:      $Y[i, j, f] \leftarrow convolve\_region(X, W, f, i, j)$ 
7:   end for
8: end for
9: return  $Y$ 
```

Dentro de la versión paralela en CPU, la optimización

se concentró únicamente en la capa convolucional. Esta decisión se tomó debido a que, según el análisis teórico y las pruebas iniciales, la convolución representa la operación de mayor costo computacional dentro de la CNN, concentrando la mayor parte del tiempo de ejecución. En comparación, etapas como ReLU, *pooling*, capa *fully connected* y *softmax* presentan una menor carga computacional relativa y, por tanto, su paralelización entrega un beneficio más reducido frente al costo adicional de implementación y sincronización.

3.4. Versión paralelizada en GPU con JAX

La implementación paralela en GPU fue desarrollada utilizando JAX, biblioteca orientada a cómputo numérico acelerado que permite ejecutar operaciones sobre GPU mediante compilación optimizada. La convolución fue encapsulada dentro de una función decorada con `@jax.jit`, lo que permite que JAX compile la operación completa antes de su ejecución y genere una versión optimizada adaptada al hardware disponible.

A diferencia de la versión CPU, donde la paralelización fue aplicada explícitamente sobre la dimensión de filtros, en GPU el paralelismo es gestionado automáticamente por JAX. La operación completa de convolución se transforma en un kernel ejecutado en paralelo sobre múltiples posiciones espaciales y filtros de manera simultánea.

Durante *forward*, la imagen de entrada se convierte a un arreglo de JAX y luego se extraen todas las ventanas de tamaño (3×3) mediante `conv_general_dilated_patches`. Posteriormente estas ventanas se reorganizan para coincidir con la estructura de los filtros y finalmente la convolución se realiza mediante `tensordot`. Cada núcleo se encarga de cada multiplicación celda a celda entre el filtro y la ventana.

Algorithm 4 Convolución paralela en GPU

Require: imagen X , filtros $F[1..N_f]$

Ensure: mapa de activación Y

```
1:  $P \leftarrow im2col(X)$ 
2: for all celda  $(i, j)$  y filtro  $f$  en paralelo do
3:    $Y[i, j, f] \leftarrow P[i, j] \cdot F[f]$ 
4: end for
5: return  $Y$ 
```

3.5. Versión distribuida con RAY

La quinta versión implementada utiliza la librería Ray para paralelizar el cálculo de la capa convolucional mediante un modelo basado en tareas (task-based parallelism). A diferencia de la versión con OpenMP, que distribuye el trabajo entre múltiples hilos dentro de un mismo proceso compartiendo memoria, Ray ejecuta tareas independientes que pueden distribuirse entre distintos procesos (e in-

cluso entre distintos nodos en un entorno distribuido). Cada tarea recibe los datos necesarios para realizar una parte del cálculo y devuelve posteriormente su resultado al proceso principal.

En esta implementación, el paralelismo se aplicó únicamente sobre la capa convolucional, ya que representa la etapa con mayor carga computacional de la red. Para ello, el conjunto de filtros convolucionales se divide en grupos, asignando cada grupo a una tarea remota de Ray.

Durante el forward pass, la imagen de entrada se almacena una única vez en el Object Store de Ray mediante la función `ray.put()`, evitando realizar múltiples copias del mismo dato para cada tarea. Posteriormente, cada worker recibe una referencia a dicha imagen y el subconjunto de filtros que le corresponde procesar. Cada tarea calcula los mapas de activación asociados a sus filtros y devuelve el resultado al proceso principal, el cual concatena todas las salidas para reconstruir el tensor completo de la convolución.

Algorithm 5 Convolución distribuida en Ray — Forward

Require: imagen X , filtros $F[1..N_f]$, W workers

Ensure: mapa de activación Y

- 1: Dividir F en W grupos: $F_1, F_2, \dots, F_W$
- 2: Publicar X en memoria compartida
- 3: **for all** worker $w \in [1..W]$ **en paralelo do**
- 4: **for all** filtro $f \in F_w$ y celda (i, j) **do**
- 5: $Y_w[i, j, f] \leftarrow X[i : i+3, j : j+3] \cdot f$
- 6: **end for**
- 7: **end for**
- 8: $Y \leftarrow \text{concatenar}(Y_1, Y_2, \dots, Y_W)$
- 9: **return** Y

4. Escenarios Experimentales y Resultados

La experimentación se centra en los tiempos de ejecución presentados por cada una de las versiones, midiendo tanto la convolución, el forward completo y el train completo. Dentro de la metodología de la experimentación, se encuentran las siguientes características:

- Dataset: CIFAR-10. Imágenes de tamaño $32 \times 32 \times 3$. Cantidad: 500, 1000, 5000 u 8000 dependiendo del experimento
- Benchmarks: tiempo de convolución, tiempo de forward completo, tiempo de train completo, energía y memoria.

Además, se utilizaron dos entornos de ejecución para los experimentos, un equipo local y un supercomputador, el Patagon, cuyas características se muestran en la Tabla 1.

Especificación	Equipo Local	Patagón (node 2)
CPU	AMD Ryzen 5 4600H	2x AMD EPYC 9534
GPU	GeForce GTX 1650Ti	RTX A4000
RAM	8GB	768GB

Table 1. Comparación especificaciones PC y patagon

Versión	Benchmark	Promedio (s)	Desv. Est.	Speedup
Python	Conv forward	53.155	0.508	1.00
NumPy	Conv forward	0.051	0.012	1048.43
Cython CPU	Conv forward	0.136	0.016	390.73
JAX GPU	Conv forward	1.069	0.107	49.73
RAY	Conv forward	11,43	0,394	4,65
Python	Forward red	58.087	0.417	1.00
NumPy	Forward red	0.100	0.016	580.98
Cython CPU	Forward red	6.379	0.249	9.11
JAX GPU	Forward red	2.070	0.188	28.07
RAY	Forward red	17,57	0,806	3,31
Python	Train completo	134.522	0.759	1.00
NumPy	Train completo	0.288	0.032	466.64
Cython CPU	Train completo	19.151	0.173	7.02
JAX GPU	Train completo	12.689	1.027	10.60
RAY	Train completo	41,21	0,677	3,263

Table 2. Resultados de rendimiento promedio para 500 imágenes y 10 repeticiones.

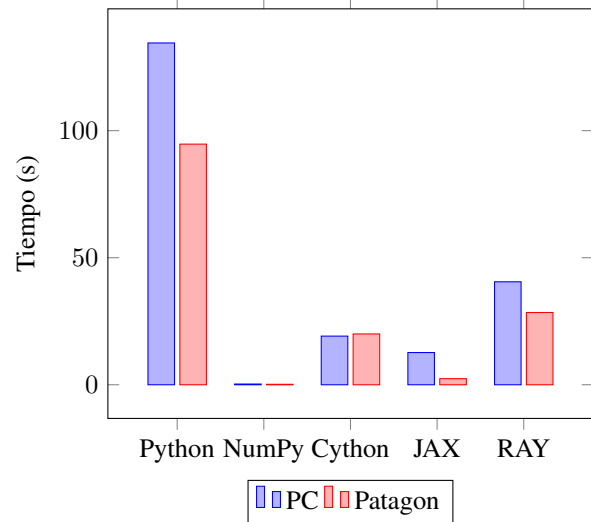


Figure 1. Comparación de tiempos de entrenamiento con 500 imágenes.

5. Discusión

Los resultados experimentales confirman que la implementación base en Python presentó el peor desempeño en todos los benchmarks evaluados. Como se observa en la Figura 1 y la Figura 2, el uso de múltiples bucles anidados para recorrer de forma secuencial las etapas de convolución, pooling y capas lineales genera una sobrecarga crítica en el

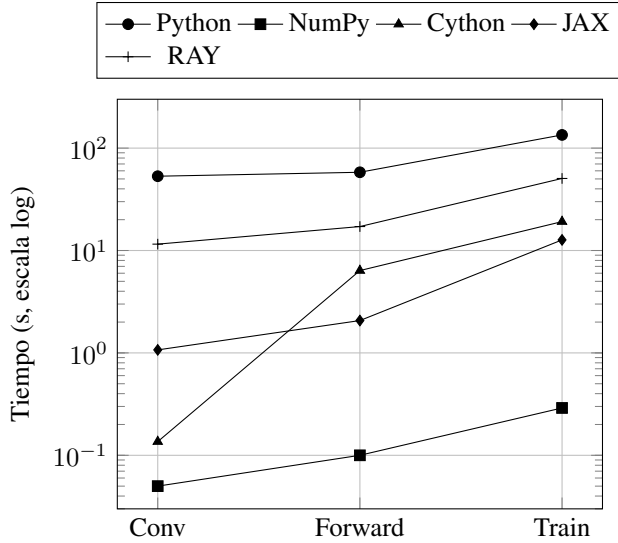


Figure 2. Comparación de tiempos promedio por etapa con 500 imágenes (Equipo local).

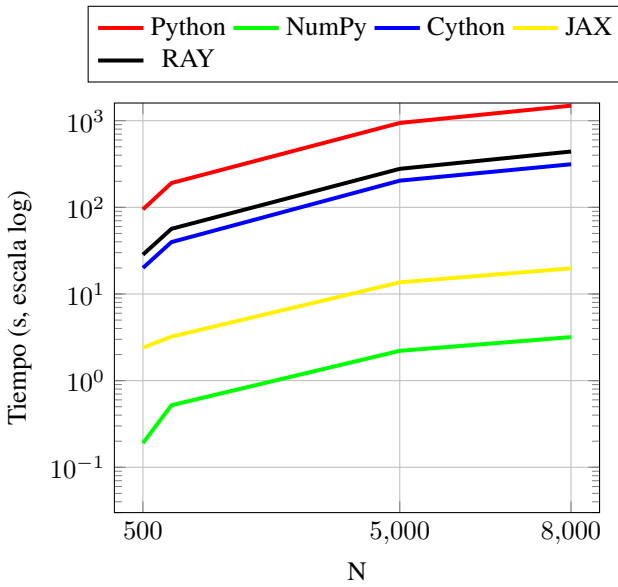


Figure 3. Comparación de tiempos de entrenamiento por cantidad de imágenes.

intérprete que degrada el rendimiento drásticamente. Esta versión sirvió estrictamente como línea base para cuantificar las aceleraciones (speedups) del resto de las arquitecturas.

La vectorización con NumPy resultó ser la optimización más efectiva, superando incluso a las alternativas paralelas. En la Figura 2 se evidencia que, para un lote de 500 imágenes, NumPy reduce el tiempo de la convolución forward a apenas 0.051 segundos (un speedup de 1048.43x). Este fenómeno se explica por la total elimi-

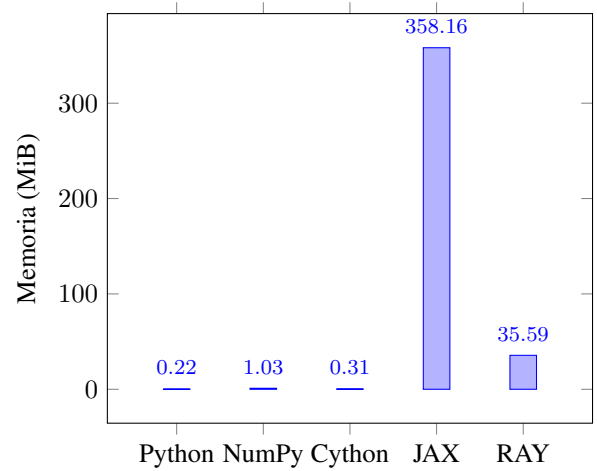


Figure 4. Comparación de peak de memoria durante ejecución.

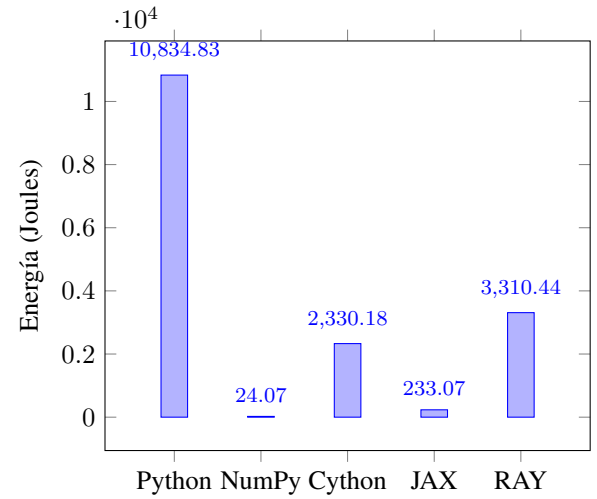


Figure 5. Comparación de energía durante ejecución.

inación de los ciclos explícitos mediante funciones nativas en C como `tensor_dot` y vistas eficientes de memoria (`sliding_window_view`), las cuales aprovechan la localidad de los datos y las instrucciones SIMD del procesador. Además, la Figura 4 y la Figura 5 demuestran que esta eficiencia matemática se traduce de forma directa en el bajo peak de memoria (1.03 MiB) y el consumo energético más bajo (24.07 Joules).

Por su parte, la paralelización en CPU mediante Cython y OpenMP logró acelerar la capa convolucional al distribuir el cálculo de los filtros en múltiples hilos independientes con la directiva `prange` y liberar el bloqueo del GIL. En entrenamiento completo, esta versión alcanzó un speedup de 7.02x respecto a la línea base. Sin embargo, al observar la Figura 2 y la Figura 3, se aprecia que el tiempo se incrementa de forma desproporcionada durante el forward global

de la red y el entrenamiento (train). Esto denota que restringir el paralelismo explícito únicamente a la dimensión de filtros limita su escalabilidad. Cuando la carga por hilo es pequeña o la cantidad de filtros es reducida (8 en esta arquitectura), el costo asociado a la creación, sincronización y comunicación de los threads reduce gran parte de la ganancia computacional.

El comportamiento de JAX en GPU (Figura 3) expone la transición hacia el paradigma de aceleración por hardware masivo. Aunque en la convolución individual de la Figura 2 sufre debido al overhead inicial de la transferencia de datos (host-to-device) y la compilación Just-In-Time (JIT) vía XLA, su rendimiento global se vuelve muy bueno a medida que el volumen de cálculo escala. En el entrenamiento completo, JAX obtuvo el mejor desempeño entre las variantes paralelas, registrando tiempos cercanos a los 12 segundos (un speedup de 10.60x). No obstante, este poder de cómputo masivo exige una contraparte crítica de recursos reflejada en las métricas físicas: la Figura 4 revela que JAX induce el pico de memoria más alto con una diferencia abismal (358.16 MiB), atribuible a la reserva del contexto de ejecución de la GPU, aunque su eficiencia energética se mantiene competitiva (233.07 Joules) gracias a la velocidad con la que finaliza el cálculo total.

Finalmente, la versión distribuida con Ray operó bajo un modelo de tareas asíncronas enfocado en la escalabilidad horizontal. A pesar de emplear el Object Store mediante `ray.put()` para mitigar la duplicación de la imagen en memoria compartida (registrando un peak moderado de 35.59 MiB en la Figura 4), Ray exhibió peores tiempos entre las opciones optimizadas, con un speedup de solo 3.26x en entrenamiento completo. Las Figuras 2, 3 y 5 confirman que el costo de serialización de datos, la latencia en la asignación de tareas a los workers distribuidos y la posterior concatenación de los mapas de activación penalizan fuertemente el rendimiento y disparan el gasto energético (3310.44 Joules). Esto demuestra que los entornos distribuidos orquestados por software no logran amortizar su infraestructura de coordinación cuando se aplican a redes neuronales de baja dimensionalidad.

En términos de escalabilidad, todas las versiones escalan de forma muy similar, sin embargo, para los casos de Python base e incluso Cython y RAY, los tiempos registrados con un N suficientemente grande se vuelven inviables (orden de 15 o 30 minutos). En cambio, las versiones de Numpy y JAX, muestran un rendimiento absolutamente manejable (no superan los 15 segundos).

6. Conclusiones

En términos generales, los resultados permiten concluir que NumPy se posiciona como la solución óptima y más eficiente de forma integral, superando a las demás alternativas no solo en tiempos de ejecución y escalabil-

idad, sino también al registrar el menor impacto en consumo energético y un buen uso de memoria. Mientras que Cython permitió implementar paralelismo explícito para aprovechar la CPU multinúcleo y JAX/GPU demostró su capacidad en volúmenes masivos de cálculo, NumPy entregó el rendimiento más balanceado y limpio al eliminar el overhead de sincronización por software o hardware. Esto confirma que la computación de alto rendimiento aplicada a redes convolucionales reduce drásticamente los tiempos de ejecución, pero que el beneficio neto final depende de la naturaleza del problema y de la eficiencia física y computacional de cada estrategia.

Como trabajo futuro sería interesante ampliar el análisis utilizando una arquitectura de mayor tamaño, aumentar el número de filtros y capas convolucionales, medir escalabilidad con distintos números de hilos, aplicar otras formas de paralelizar (por batches o por fila de la imagen), o comparar el rendimiento con librerías especializadas como PyTorch o TensorFlow. Esto permitiría evaluar con mayor profundidad el comportamiento de cada estrategia en escenarios más cercanos a aplicaciones reales de deep learning.

7. Anexo

Todo el detalle de la implementación se encuentra en el repositorio de GitHub: <https://github.com/AndresMardonesUACH/CNN-optimization-with-HPC-tools>

References

- [1] Stefan Behnel, Robert Bradshaw, Craig Citro, Lisandro Dalcin, Dag Sverre Seljebotn, and Kurt Smith. Cython: The best of both worlds. *Computing in Science & Engineering*, 13(2):31–39, 2011. 2
- [2] James Bradbury, Roy Frostig, Peter Hawkins, Chris James, Matthew Jones, Robert Kirchoff, Matthew Tancik, Jake VanderPlas, Roman Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. 2
- [3] Kumar Chellapilla, Siddhaman Puri, and Patrice Simard. High performance convolutional neural networks for document processing. In *Tenth International Workshop on Frontiers in Handwriting Recognition*. Suvisoft, 2006. 1
- [4] Shahrin Rahman. Optimizing cnn using hpc tools. *arXiv preprint arXiv:2403.04870*, 2024. 2
- [5] W. Zhao et al. An fpga-based framework for training convolutional neural networks. *Journal of Real-Time Image Processing*, 21(2):45–58, 2024. 2